

Redukcja problemu MIN-BED do SAT oraz jej implementacja

(Reduction of the MIN-BED Problem to SAT and Its Implementation)

Wojciech Kieloch

Praca inżynierska

Promotor: dr hab. Jakub Michaliszyn, prof. UWr

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

2 września 2026

Streszczenie

W pracy przedstawiono praktyczną implementację rozwiązania problemu MIN-BED, dotyczącego minimalizacji automatów skończonych z uwzględnieniem dystansu edycyjnego. Zaproponowano autorską metodę kodowania problemu do postaci SAT, opartą na wcześniej opracowanym algorytmie. Uzyskane wyniki pokazują, że problem pozostaje trudny do rozwiązania również w praktyce, jednak opracowana metoda stanowi podstawę do dalszego rozwoju i optymalizacji badanego rozwiązania.

Abstract

This paper presents a practical implementation of a solution to the MIN-BED problem, which concerns the minimization of finite automata modulo a bounded edit distance. An original encoding of the problem into SAT is presented, based on a previously developed algorithm. The results show that the problem remains difficult to solve in practice as well; however, the developed method provides a foundation for further development and optimization of the solution under study.

Spis treści

1. Wstęp	7
1.1. Motywacja	7
1.2. Cel pracy	8
1.3. Stan wiedzy	8
1.4. Istniejące rozwiązania	9
2. Preliminaria	11
2.1. Podstawowe definicje	11
2.2. Dystans edycyjny	12
2.3. Problem MIN-BED	13
2.4. Algorytm	13
3. Opis redukcji	15
3.1. Kodowanie	15
3.1.1. Prekalkulacja	16
3.1.2. Wstępna uwaga na temat analizy asymptotycznej	17
3.1.3. Etap I: Zgadywanie automatu $\mathcal{B}$	18
3.1.4. Etap II: Budowa $\text{dag}(B)$	21
3.1.5. Etap III: Covers	22
3.1.6. Etap IV: Warunek $\mathcal{B} \subseteq_{\text{ED}}^{\infty} \mathcal{A}$	24
3.1.7. Etap V: Domknięcie $\text{dag}(B)$	26
3.1.8. Etap VI: Warunek $\mathcal{A} \subseteq_{\text{ED}}^{\infty} \mathcal{B}$	27
3.1.9. Analiza asymptotyczna	29

3.1.10. Certyfikat	30
3.1.11. Dyskusja o poprawności i zupełności	30
3.2. Heurystyka generowania automatów o zadanych parametrach	32
3.2.1. Algorytm	32
3.2.2. Własności wygenerowanego automatu	34
3.2.3. Ograniczenia metody	34
4. Implementacja	35
4.1. Zastosowane technologie	35
5. Wyniki	37
5.1. Rozmiar formuły SAT	38
5.2. UNSAT	40
5.3. Instancje optymalne i graniczne	41
5.4. Wnioski	44
6. Podsumowanie	45
Bibliografia	47
A Załącznik - szczegóły implementacji	49
A.1. Architektura systemu	49
A.1.1. Struktura projektu	49
A.1.2. Programy użytkowe	50
A.1.3. Format pliku DFA	51
A.2. Sposób użytkowania	52

Rozdział 1.

Wstęp

Automaty skończone stanowią jeden z podstawowych modeli obliczeń, odgrywając istotną rolę zarówno w informatyce teoretycznej, jak i w licznych zastosowaniach praktycznych.

Jednym z fundamentalnych problemów związanych z deterministycznymi automatami skończonymi (DFA) jest ich minimalizacja. W klasycznym ujęciu polega ona na redukcji liczby stanów automatu przy jednoczesnym zachowaniu równoważności językowej, a tym samym na zmniejszeniu złożoności modelu.

Niniejsza praca obejmuje zarówno teoretyczne opracowanie stanowiące podstawę implementacji, jak i samą implementację wariantu problemu minimalizacji, w którym dopuszcza się kontrolowane zmiany rozpoznawanego języka w celu uzyskania większej redukcji rozmiaru automatu.

1.1. Motywacja

Automaty wykorzystywane są między innymi w rozpoznawaniu wzorców [5], systemach opartych na regułach [7], planowaniu [8], rozpoznawaniu języków [9] oraz systemach wieloagentowych [10]. Szczególne znaczenie wśród automatów skończonych mają deterministyczne automaty skończone (ang. *Deterministic Finite Automata*), które stanowią jeden z najprostszych formalizmów pozwalających na reprezentowanie i analizowanie struktur, wzorców, reguł oraz procesów.

W przypadku modeli uczonych na danych rzeczywistych wejścia mogą być zaśsumione, niepełne lub obciążone innymi niedoskonałościami. W takich warunkach dokładne zachowanie modelu nie zawsze musi być konieczne, a przybliżenie jego działania może być wystarczające dla danego zastosowania. Z tego względu zachowanie dokładnej równoważności języków podczas minimalizacji DFA nie zawsze jest adekwatnym rozwiązaniem. Bardziej odpowiednie może okazać się zastosowanie pojęcia podobieństwa języków, które pozwala zaakceptować niewielkie różnice pomiędzy ję-

zykiem oryginalnego automatu a językiem automatu zminimalizowanego, jednocześnie umożliwiając uzyskanie większej redukcji liczby stanów.

Istotne znaczenie w tym kontekście ma odległość edycyjna. Określa ona minimalną liczbę operacji wstawienia, usunięcia lub zastąpienia pojedynczego znaku, które są niezbędne do przekształcenia jednego słowa w drugie. Odległość ta jest szeroko stosowana jako miara podobieństwa ciągów znaków, między innymi w przetwarzaniu języka naturalnego, analizie DNA oraz analizie sieci neuronowych.

Zastosowanie odległości edycyjnej do języków formalnych pozwala na bardziej elastyczne podejście do minimalizacji deterministycznych automatów skończonych. Zamiast wymagać zachowania dokładnie tego samego języka, można dopuścić kontrolowane różnice pomiędzy językiem wejściowego automatu a językiem automatu zminimalizowanego. Pozwala to na kompromis pomiędzy rozmiarem automatu a stopniem jego zgodności z pierwotnym modelem.

Wariant tego typu minimalizacji jest przedmiotem niniejszej pracy.

1.2. Cel pracy

Celem jest zbadanie praktycznej możliwości rozwiązania problemu minimalizacji deterministycznych automatów skończonych z uwzględnieniem odległości edycyjnej, konkretnie problemu określonego w literaturze jako MIN-BED.

W ramach pracy opracowano autorską metodę kodowania problemu do postaci instancji SAT, opartą na algorytmie przedstawionym we wcześniejszej pracy. Zaimplementowano zaproponowane rozwiązanie oraz przeprowadzono eksperymentalną analizę jego działania z wykorzystaniem wyspecjalizowanych solverów SAT.

Celem eksperymentów jest ocena praktycznej trudności problemu oraz określenie możliwości jego efektywnego rozwiązania. Uzyskane wyniki stanowią również podstawę do wskazania potencjalnych kierunków dalszego rozwoju i optymalizacji opracowanej metody, a implementacja baze do tworzenia kolejnych rozwiązań w dziedzinie.

1.3. Stan wiedzy

Różne pojęcia podobieństwa języków formalnych prowadzą do odmiennych wariantów problemu minimalizacji automatów skończonych. Klasyczna minimalizacja DFA zakłada zachowanie dokładnie tego samego języka przy jednoczesnym dążeniu do uzyskania automatu o minimalnej liczbie stanów. Dla automatu o n stanach istnieją algorytmy realizujące ten problem w czasie $O(n \log n)$, co czyni klasyczną minimalizację jednym z podstawowych i dobrze poznanych problemów teorii automatów.

W literaturze rozważano również bardziej elastyczne pojęcia równoważności i podobieństwa języków. Przykładowo, dwa języki można uznać za podobne, jeżeli ich różnica symetryczna jest skończona. Minimalizacja DFA przy takim założeniu nazywana jest hiperminimalizacją [2]. Innym podejściem jest wykorzystanie dystansu pomiędzy słowami i rozszerzenie go na języki. Dla danego słowa w oraz języka L definiuje się wówczas $d(w, L)$ jako najmniejszą odległość między w a dowolnym słowem należącym do L . Pozwala to porównywać języki również wtedy, gdy dopuszczamy niewielkie modyfikacje należących do nich słów.

Praca J. Michaliszyna i J. Otopa [1] analizuje warianty problemu minimalizacji, w którym dopuszcza się skończone różnice w dystansie edycyjnym języków. Autorzy badają między innymi złożoność problemu w zależności od liczby dopuszczalnych modyfikacji. Wykazują, że wariant, w którym górna wymagana liczba modyfikacji jest określona jako część ograniczenia problemu, jest PSPACE-zupełny, natomiast wariant z ograniczoną, lecz nieustaloną liczbą modyfikacji (MIN-BED) należy do klasy Σ_2^P . Po nałożeniu dodatkowego ograniczenia ten ostatni okazuje się problemem NP-zupełnym. To właśnie ten punkt stanowi wyjście dla niniejszej pracy.

1.4. Istniejące rozwiązania

Problem MIN-BED nie został dotychczas opracowany od strony praktycznej. Wyniki dotyczą przede wszystkim teoretycznych własności problemu oraz jego złożoności obliczeniowej dla różnych wariantów. Brakuje rozwiązań, które można byłoby przebadать w praktyce.

Również w przypadku hiperminimalizacji, mimo istnienia wyników teoretycznych opisujących problem i jego własności, autor nie znalazł przykładów jej praktycznego zastosowania ani powszechnie dostępnych implementacji, które mogłyby stanowić punkt odniesienia dla projektowanego rozwiązania. Wskazuje to na różnicę pomiędzy stopniem rozwoju teoretycznego tych metod a ich wykorzystaniem w praktyce.

Znacznie lepiej opracowany jest natomiast problem klasycznej minimalizacji deterministycznych automatów skończonych. Ze względu na prostotę, efektywność oraz wieloletnie stosowanie stała się ona standardowym elementem pracy z automatami skończonymi i jest powszechnie wykorzystywana w praktyce.

Rozdział 2.

Preliminaria

2.1. Podstawowe definicje

Słowo w nad skończonym alfabetem Σ jest skończonym ciągiem symboli należących do Σ . Zbiór wszystkich skończonych słów nad alfabetem Σ oznaczamy przez Σ^* . Symbolem ε oznaczamy słowo puste.

Deterministyczny automat skończony (DFA, ang. *Deterministic Finite Automaton*) jest piątką $A = \langle \Sigma, Q, q_0, F, \delta \rangle$, gdzie Σ jest skończonym alfabetem, Q jest skończonym zbiorem stanów, $q_0 \in Q$ jest stanem początkowym, $F \subseteq Q$ jest zbiorem stanów akceptujących, a $\delta : Q \times \Sigma \rightarrow Q$ jest funkcją przejścia. Rozmiar automatu A , oznaczany przez $|A|$, jest równy liczbie jego stanów: $|A| = |Q|$.

Funkcję przejścia δ rozszerzamy na wszystkie słowa z Σ^* poprzez następujące definicje:

$$\begin{aligned}\delta(q, \varepsilon) &= q, \\ \delta(q, wx) &= \delta(\delta(q, w), x),\end{aligned}$$

dla każdego stanu $q \in Q$, słowa $w \in \Sigma^*$ oraz symbolu $x \in \Sigma$.

Dla słowa $w = w_1 \dots w_n$ przebieg automatu A na słowie w jest ciągiem stanów $q_0 q_1 \dots q_n$, takim że q_0 jest stanem początkowym oraz dla każdego $i \in \{1, \dots, n\}$ zachodzi $\delta(q_{i-1}, w_i) = q_i$.

Przebieg nazywamy akceptującym, jeżeli jego ostatni stan należy do zbioru stanów akceptujących, tj. $q_n \in F$. Językiem automatu A , oznaczanym przez $L(A)$, jest zbiór wszystkich słów, dla których przebieg automatu jest akceptujący: $L(A) = \{w \in \Sigma^* \mid \delta(q_0, w) \in F\}$.

Podzbiór stanów $C \subseteq Q$ deterministycznego automatu A nazywamy maksymalną silnie spójną składową (SCC, ang. *Strongly Connected Component*), jeżeli dla każdych dwóch stanów $s_1, s_2 \in C$ istnieje niepuste słowo w takie, że $\delta(s_1, w) = s_2$, oraz nie istnieje zbiór $C' \subseteq Q$ taki, że $C \subseteq C'$ i $C \neq C'$, dla którego powyższa własność również zachodzi.

Mówimy, że automat A ma głębokość SCC równą c , jeżeli c jest maksymalną liczbą różnych maksymalnych silnie spójnych składowych odwiedzanych przez akceptujący przebieg automatu.

Przez $\text{SCC}(A)$ oznaczamy zbiór wszystkich maksymalnych silnie spójnych składowych automatu A , które zawierają stan akceptujący lub z których stan akceptujący jest osiągalny.

Definiujemy $\text{dag}(A)$ jako skierowany graf acykliczny, którego zbiór wierzchołków stanowi $\text{SCC}(A)$. Graf ten zawiera krawędź z C_1 do C_2 wtedy i tylko wtedy, gdy $C_1 \neq C_2$ oraz istnieje przejście ze stanu należącego do C_1 do stanu należącego do C_2 . Niech $\text{dag}^*(A)$ oznacza zwrotne i przechodnie domknięcie $\text{dag}(A)$.

Dla automatów $\mathcal{A}$ i $\mathcal{B}$ oraz ich składowych C i D , definiujemy notację:

$$\mathcal{L}(\mathcal{A}|C) \subseteq \mathcal{L}(\mathcal{B}|D) \iff \exists q \in C, r \in D \quad \mathcal{L}(\mathcal{A}|_q C) \subseteq \mathcal{L}(\mathcal{B}|_r D)$$

gdzie $\mathcal{L}(\mathcal{A}|_q C)$ reprezentuje język automatu $\mathcal{A}$ ze stanem początkowym q , ograniczonego do stanów składowej C , w którym wszystkie stany są akceptujące, a wyjście poza składową oznacza przejście do stanu odrzucającego (ang. *sink*).

Rozważmy dwa automaty DFA $\mathcal{A}$ i $\mathcal{B}$. Mówimy, że ścieżka $\pi = C_1 \dots C_n$ w $\text{dag}(\mathcal{A})$ jest pokryta przez ścieżkę $\pi' = D_1 \dots D_m$ w $\text{dag}^*(\mathcal{B})$ wtedy i tylko wtedy, gdy $n = m$ oraz dla każdego $1 \leq i \leq n$ zachodzi $\mathcal{L}(\mathcal{A}|_{C_i}) \subseteq \mathcal{L}(\mathcal{B}|_{D_i})$.

2.2. Dystans edycyjny

Dystans edycyjny pomiędzy słowami w i w' , oznaczany przez $\text{ED}(w, w')$, jest minimalną liczbą operacji potrzebnych do przekształcenia słowa w w słowo w' . Dopuszczalnymi operacjami są: wstawienie symbolu, usunięcie symbolu oraz zamiana jednego symbolu na inny. Przykładowo $\text{ED}(ab, acd) = 2$, ponieważ do przekształcenia słowa ab w acd wystarczają jedna zamiana symbolu oraz jedno wstawienie.

Dla liczby naturalnej k mówimy, że język L jest zawarty w języku L' z dokładnością/modulo do dystansu edycyjnego k , co oznaczamy przez $L \subseteq_{\text{ED}}^k L'$, jeżeli dla każdego słowa $w \in L$ istnieje słowo $w' \in L'$ takie, że $\text{ED}(w, w') \leq k$. Definicję tę rozszerzamy na $k = \infty$. Mówimy, że $L \subseteq_{\text{ED}}^\infty L'$, jeżeli istnieje liczba naturalna k taka, że $L \subseteq_{\text{ED}}^k L'$.

Dwa deterministyczne automaty skończone A i B nazywamy równoważnymi z dokładnością do dystansu edycyjnego $k \in \mathbb{N} \cup \{\infty\}$, co oznaczamy przez $A \equiv_{\text{ED}}^k B$, jeżeli ich języki spełniają jednocześnie $L(A) \subseteq_{\text{ED}}^k L(B)$ oraz $L(B) \subseteq_{\text{ED}}^k L(A)$.

2.3. Problem MIN-BED

Problem minimalizacji automatu modulo ograniczony dystans edycyjny (MIN-BED, ang. *Minimization modulo a Bounded Edit Distance*) definiuje się następująco:

Wejście: deterministyczny automat skończony A oraz liczba naturalna m ;

Wyjście: czy istnieje deterministyczny automat skończony B mający co najwyżej m stanów taki, że $A \equiv_{\text{ED}}^\infty B$?

Celem problemu MIN-BED jest ustalenie, czy dla danego automatu A istnieje automat B o co najwyżej m stanach, którego język jest równoważny językowi A w sensie ograniczonego dystansu edycyjnego.

Twierdzenie 1. *Dla każdego $c \geq 3$ problem MIN-BED ograniczony do deterministycznych automatów skończonych o głębokości SCC co najwyżej c jest NP-zupełny [1, Theorem 8].*

2.4. Algorytm

Algorytm decydujący, czy dla danych deterministycznych automatów skończonych A i B zachodzi $A \equiv_{\text{ED}}^\infty B$, zakłada, że liczba ścieżek w $\text{dag}(A)$ jest ograniczona przez $|A|^l$, gdzie $l \in \mathbb{N}$ jest stałą, którą można interpretować jako głębokość SCC. Nie nakłada się ograniczenia na liczbę ścieżek w $\text{dag}(B)$ [1, Lemma 5].

Aby sprawdzić, czy zachodzi $A \subseteq_{\text{ED}}^\infty B$, dla każdej ścieżki w $\text{dag}(A)$ algorytm niedeterministycznie wyszukuje ścieżkę w $\text{dag}^*(B)$, która ją pokrywa.

Sprawdzenie warunku $B \subseteq_{\text{ED}}^\infty A$ jest bardziej złożone. Niech Π oznacza zbiór wszystkich ścieżek w $\text{dag}(A)$.

Dla każdego wierzchołka C grafu $\text{dag}(B)$ algorytm niedeterministycznie wybiera niepusty podzbiór $\text{covers}(C) \subseteq \Pi$ taki, że dla każdej ścieżki $\pi \in \text{covers}(C)$ jej ostatni wierzchołek D spełnia warunek $L(B|C) \subseteq L(A|D)$.

Dla wierzchołków C, C' takich, że C' jest następnikiem C w $\text{dag}(B)$, oraz dla ścieżki $\pi \in \text{covers}(C)$ mówimy, że ścieżka π' jest *kontynuacją* π w C' , jeżeli $\pi' \in \text{covers}(C')$ oraz π jest prefiksem π' lub $\pi = \pi'$.

Algorytm akceptuje wtedy i tylko wtedy, gdy dla każdego wierzchołków C, C' takich, że C' jest następnikiem C w $\text{dag}(B)$, każda ścieżka $\pi \in \text{covers}(C)$ posiada kontynuację należącą do $\text{covers}(C')$.

Rozdział 3.

Opis redukcji

3.1. Kodowanie

Poniżej przedstawiono redukcję problemu MIN-BED do problemu SAT. Formuły, których nie można bezpośrednio zapisać w postaci CNF, mogą zostać przekształcone do równospelnialnych formuł CNF za pomocą transformacji Tseytina [4]. Zostanie to uwzględnione przy analizie asymptotycznego rozmiaru kodowania.

Kodowanie wykorzystuje zmienne binarne i jest formułowane za pomocą ograniczeń pseudoboolowskich. W przypadku wyboru jednego elementu spośród n możliwości zamiast reprezentacji unarnej, wymagającej $O(n)$ zmiennych, zastosowana zostaje reprezentacja bitowa wykorzystująca $O(\log n)$ zmiennych.

W zapisie kodowania wykorzystywane są operatory $\leq$ i $=$ do określania relacji pomiędzy zmienną a stałą. W praktycznej implementacji wystarczające jest jednak wykorzystanie operatorów $<$ oraz $\neq$. Nierówność ma przy tym szczególne znaczenie, ponieważ jej naturalna postać może zostać bezpośrednio wyrażona w CNF, co ma istotny wpływ na wydajność rozwiązywania zakodowanego problemu.

Rozmiar kodowania analizowany będzie w dwóch aspektach: pod względem liczby zmiennych oraz liczby wystąpień literalów. Przez liczbę wystąpień literalów rozumiana jest łączna liczba wystąpień zmiennych, wraz z ich negacjami, we wszystkich klauzulach formuły. Przyjmujemy, że zapisanie równości wymaga liniowej liczby literalów i nie wprowadza dodatkowych zmiennych. W przypadku relacji „mniejsze niż” zakładamy natomiast liniową liczbę literalów oraz liniową liczbę zmiennych pomocniczych.

Kodowanie zostało podzielone na sześć etapów. Każdy z nich przedstawia nowo wprowadzane zmienne oraz reguły kodowania, a także zawiera analizę asymptotycznego rozmiaru powstałej formuły. Etapy 1–2 stanowią część wspólną kodowania, etapy 3–4 kodują warunek $B \subseteq_{\text{ED}}^{\infty} A$, a etapy 5–6 kodują warunek $A \subseteq_{\text{ED}}^{\infty} B$.

3.1.1. Prekalkulacja

Przed przystąpieniem do kodowania należy wykonać prekalkulację na podanym na wejściu automacie. Każdą z nich można wykonać w wielomianowej pamięci i złożoności.

Należy:

1. Ponumerować stany automatu w zbiorze $\{1, \dots, |A|\}$ oraz symbole jego alfabetu w zbiorze $\{1, \dots, |E|\}$.
2. Obliczyć DAG automatu zgodnie z definicją przedstawioną w pracy.
3. Ponumerować wierzchołki grafu DAG w zbiorze $\{1, \dots, |V(\text{dag}(A))|\}$.
4. Ponumerować wszystkie ścieżki grafu DAG automatu.

Stałe w kodowaniu

W kodowaniu wykorzystywane są następujące stałe:

- $\text{sink}_A = |A| + 1$ – reprezentuje abstrakcyjny stan automatu A , którego nie można opuścić.
- $\text{sink}_B = m + 1$ – reprezentuje abstrakcyjny stan automatu B , którego nie można opuścić.
- PathsCount – liczba wszystkich ścieżek w automacie.

Funkcje pomocnicze

Do implementacji kodowania potrzebne będą następujące funkcje:

- $\text{NextState}(q, a)$ – zwraca $\delta(q, a)$, jeżeli $q \neq \text{sink}_A$ oraz $\delta(q, a)$ należy do tej samej składowej SCC co q ; w przeciwnym przypadku zwraca sink_A .
- $\text{PathLen}(p)$ – zwraca długość ścieżki o identyfikatorze p .
- $\text{LastNode}(p)$ – zwraca identyfikator ostatniego wierzchołka na ścieżce o identyfikatorze p .
- $\text{SccNodes}(d)$ – zwraca zbiór identyfikatorów stanów należących do składowej d .
- $\text{IsPrefix}(p_1, p_2)$ – sprawdza, czy ścieżka o identyfikatorze p_1 jest prefiksem ścieżki o identyfikatorze p_2 .
- $\text{PathNode}(p, i)$ – zwraca numer i -tego wierzchołka na ścieżce o numerze p .

Jedyną funkcją wymagającą dodatkowego komentarza jest `IsPrefix`. Należy zauważyć, że maksymalna głębokość składowych w automacie A wynosi l , a zatem każda ścieżka może zawierać co najwyżej l wierzchołków.

3.1.2. Wstępna uwaga na temat analizy asymptotycznej

Zgodnie z założeniami algorytmu przyjmuje się, że liczba ścieżek w DAG-u automatu A jest ograniczona przez $O(|A|^l)$, gdzie $l \in \mathbb{N}$. Dodatkowo zakłada się, że $l \geq 3$, ponieważ dla mniejszych wartości istnieją skuteczne algorytmy rozwiązujące rozważany problem [1, Algorithm 1]. Przyjmuje się również, że $|A| > m > 0$, gdzie m oznacza maksymalną liczbę stanów poszukiwanego automatu – w przeciwnym przypadku odpowiedź jest trywialna.

Podsumowanie rozmiaru kodowania oraz dyskusja na jego temat zostały przedstawione w dalszej części pracy. Należy jednak zwrócić uwagę, że ze względu na występowanie w algorytmie trzech kwantyfikatorów uniwersalnych, po których następuje jeden kwantyfikator egzystencjalny, oczekiwana liczba wystąpień literalów wynosi $\Omega(m^2|A|^l)$. Warto zauważyć, że $O(m^2|A|^l)$ dominuje funkcje $O(m^k)$ dla $k < l + 2$.

3.1.3. Etap I: Zgadywanie automatu $\mathcal{B}$

Znajdowanie DFA o co najwyżej m stanach.

Zmienne

Zakresy indeksów:

$$q, p, s \in \{1, \dots, m\},$$

$$a \in \{1, \dots, |\Sigma|\}.$$

Zmienna	W etapach	Opis
$t_{q,a,p}$	III, VI	Istnieje przejście z q do p za pomocą litery a .
$origin_q$	Lokalna	Stan q jest startowy.
$final_q$	Lokalna	Stan q jest akceptujący.
$canReachFinal_q$	II	Ze stanu q można dojść do stanu akceptującego.
$active_q$	Lokalna	Stan q jest używany w automacie (zmienna pomocnicza).
$e_{p,q}$	Lokalna	Można przejść jedną literą ze stanu p do stanu q .
$r_{p,q,s}$	Lokalna	Stan q jest osiągalny ze stanu p w kroku s .
$wr_{p,q}$	II	Stan q jest osiągalny słowem ze stanu p .

Tabela 3.1: Zmienne

Reguły

Przynajmniej jeden stan jest stanem początkowym.

$$\bigvee_{q=1}^m origin_q \quad (1a)$$

Co najwyżej jeden stan jest stanem początkowym.

$$\bigwedge_{1 \leq q_1 < q_2 \leq m} (origin_{q_1} \implies \neg origin_{q_2}) \quad (1b)$$

Przynajmniej jeden stan jest stanem finalnym.

$$\bigvee_{q=1}^m \text{final}_q \quad (1c)$$

Dla każdej pary indeksów q i a istnieje co najwyżej jedno takie p , że $t_{q,a,p}$ jest prawdziwe (inaczej: funkcja przejścia jest jednoznaczna).

$$\bigwedge_{q=1}^m \bigwedge_{a=1}^{|\Sigma|} \bigwedge_{1 \leq p_1 < p_2 \leq m} (t_{q,a,p_1} \implies \neg t_{q,a,p_2}) \quad (1d)$$

Jeżeli istnieje przejście między stanami q a p , to oba te stany są aktywne.

$$\bigwedge_{q=1}^m \bigwedge_{a=1}^{|\Sigma|} \bigwedge_{p=1}^m (t_{q,a,p} \implies \text{active}_q \wedge \text{active}_p) \quad (1e)$$

Jeżeli stan q jest aktywny, to w funkcji przejścia ma zdefiniowane wszystkie wyjścia.

$$\bigwedge_{q=1}^m \bigwedge_{a=1}^{|\Sigma|} \left(\text{active}_q \implies \bigvee_{p=1}^m t_{q,a,p} \right) \quad (1f)$$

Jeżeli stan jest początkowy lub finalny, to jest aktywny.

$$\bigwedge_{q=1}^m (\text{origin}_q \vee \text{final}_q \implies \text{active}_q) \quad (1g)$$

Jeżeli stan jest aktywny, to musi być osiągalny z początku.

$$\bigwedge_{q=1}^m \left(\text{active}_q \implies \bigvee_{p=1}^m (\text{origin}_p \wedge r_{p,q,m}) \right) \quad (1h)$$

Istnieje krawędź między p i q , jeżeli jakieś przejście tak prowadzi.

$$\bigwedge_{p=1}^m \bigwedge_{q=1}^m \left(e_{p,q} \iff \bigvee_{a=1}^{|\Sigma|} t_{p,a,q} \right) \quad (1i)$$

Możliwość dojścia do stanu finalnego.

$$\bigwedge_{q=1}^m \left(\text{canReachFinal}_q \iff \bigvee_{p=1}^m (r_{q,p,m} \wedge \text{final}_p) \right) \quad (1j)$$

Kodowanie osiągalności

Każdy stan jest osiągalny sam z siebie w pierwszym kroku.

$$\bigwedge_{q=1}^m r_{q,q,1} \quad (1k)$$

Żaden stan nie jest osiągalny w jednym kroku z innego stanu niż on sam.

$$\bigwedge_{\substack{1 \leq p, q \leq m \\ p \neq q}} \neg r_{p,q,1} \quad (1l)$$

Jeżeli (i, j) jest osiągalne w czasie k , to również w późniejszym. Jeżeli istnieje krawędź (i, l) , a j jest osiągalne przez l w czasie k , to i osiąga j w kolejnym kroku.

$$\bigwedge_{i=1}^m \bigwedge_{j=1}^m \bigwedge_{k=1}^{m-1} \left(r_{i,j,k+1} \iff r_{i,j,k} \vee \bigvee_{l=1}^m (e_{i,l} \wedge r_{l,j,k}) \right) \quad (1m)$$

Na potrzeby SCC określamy osiągalność słowem. Stan jest osiągalny sam z siebie wtedy i tylko wtedy, gdy ma pętlę lub istnieje cykl go zawierający.

$$\bigwedge_{p=1}^m \left(wr_{p,p} \iff \bigvee_{a=1}^{|\Sigma|} t_{p,a,p} \vee \bigvee_{q=1}^m (e_{p,q} \wedge r_{q,p,m}) \right) \quad (1n)$$

$$\bigwedge_{\substack{1 \leq p, q \leq m \\ p \neq q}} (wr_{p,q} \iff r_{p,q,m}) \quad (1o)$$

Rozmiar

Podstawowe zmienne są asymptotycznie zdominowane przez zmienne t i r , których łączna liczba wynosi $O(m^2|\Sigma| + m^3)$. Reguły (1h), (1j), (1m) i (1n) wymagają zastosowania transformacji Tseytina. Liczbę zmiennych pomocniczych wprowadzanych w tej transformacji asymptotycznie dominuje reguła (1m), która prowadzi do utworzenia $O(m^4)$ dodatkowych zmiennych. Łączna liczba zmiennych wynosi zatem $O(m^2|\Sigma| + m^4)$. Liczba wystąpień literalów jest asymptotycznie zdominowana przez reguły (1d) i (1m) i wynosi $O(m^3|\Sigma| + m^4)$.

3.1.4. Etap II: Budowa $\text{dag}(B)$

Budowanie grafu SCC automatu B .

Zmienne

Zakresy indeksów: $q, p, c, d \in \{1, \dots, m\}$

Zmienna	W etapach	Opis
$belongs_{c,q}$	III, V, VI	Stan q należy do SCC o numerze c .
$dagE_{c,d}$	IV, V	Istnieje krawędź między SCC c oraz d .
$sameComponent_{q,p}$	III, VI	Stany q i p należą do tego samego SCC.
$compIsNotEmpty_c$	III	Czy jakiś stan należy do SCC o identyfikatorze c .

Tabela 3.2: Zmienne

Reguły

Jeśli q należy do komponentu c , to nie należy do żadnego innego.

$$\bigwedge_{q=1}^m \bigwedge_{c=1}^m \bigwedge_{\substack{k=1 \\ k \neq c}}^m (belongs_{c,q} \implies \neg belongs_{k,q}) \quad (2a)$$

Jeżeli stany należą do tego samego komponentu, to muszą być wzajemnie osiągalne i da się z nich osiągnąć stan akceptujący.

$$\bigwedge_{c=1}^m \bigwedge_{q=1}^m \bigwedge_{p=1}^m (belongs_{c,q} \wedge belongs_{c,p} \implies wr_{q,p} \wedge wr_{p,q} \wedge canReachFinal_q) \quad (2b)$$

Jeżeli stany są wzajemnie osiągalne i da się z nich osiągnąć stan akceptujący, to należą razem do jakiegoś komponentu.

$$\bigwedge_{p=1}^m \bigwedge_{q=1}^m \left(wr_{p,q} \wedge wr_{q,p} \wedge canReachFinal_q \implies \bigvee_{c=1}^m (belongs_{c,p} \wedge belongs_{c,q}) \right) \quad (2c)$$

Jeżeli dwa stany są osiągalne słowem, a ich komponenty są różne, to w DAG-u istnieje krawędź.

$$\bigwedge_{p=1}^m \bigwedge_{q=1}^m \bigwedge_{c_1=1}^m \bigwedge_{\substack{c_2=1 \\ c_2 \neq c_1}}^m (wr_{p,q} \wedge belongs_{c_1,p} \wedge belongs_{c_2,q} \implies dagE_{c_1,c_2}) \quad (2d)$$

Jeżeli w DAG-u istnieje krawędź, to istnieje przejście między komponentami.

$$\bigwedge_{c_1=1}^m \bigwedge_{\substack{c_2=1 \\ c_2 \neq c_1}}^m \left(dagE_{c_1,c_2} \implies \bigvee_{p=1}^m \bigvee_{q=1}^m (belongs_{c_1,p} \wedge belongs_{c_2,q} \wedge wr_{p,q}) \right) \quad (2e)$$

Dwa stany należą do tego samego komponentu wtw. gdy są wzajemnie osiągalne słowem i da się z nich osiągnąć stan akceptujący.

$$\bigwedge_{p=1}^m \bigwedge_{q=1}^m (sameComponent_{p,q} \iff rw_{p,q} \wedge rw_{q,p} \wedge canReachFinal_q) \quad (2f)$$

Komponent jest niepusty wtw. gdy jakiś stan do niego należy.

$$\bigwedge_{c=1}^m \left(compIsNotEmpty_c \iff \bigvee_{q=1}^m belongs_{c,q} \right) \quad (2g)$$

Rozmiar

Podstawowe zmienne są asymptotycznie zdominowane przez zmienne *belongs*, *dagE* i *sameComponent*, których łączna liczba wynosi $O(m^2)$. Reguły (2c) i (2e) wymagają zastosowania transformacji. Reguła (2e) wprowadza dodatkowo $O(m^4)$ zmiennych pomocniczych. Łączna liczba zmiennych wynosi zatem $O(m^4)$. Liczba wystąpień literałów jest asymptotycznie zdominowana przez reguły (2d) i (2e) i wynosi $O(m^4)$.

3.1.5. Etap III: Covers

Przypisywanie ścieżek do zbiorów. Tworzony jest jeden iloczyn automatów. Jeżeli solver chce przypisać ścieżkę kończącą się na SCC D do odpowiedniego zbioru $covers(C)$, musi wybrać taki stan startowy (p, q) , że p należy do komponentu C , a q do komponentu D . Przy czym propagacja nie może skutkować osiągnięciem stanu, który dowodzi, że istnieje słowo akceptowane w zakresie C , ale nieakceptowane w D (kontrprzykład zawierania).

Zmienne

Zakresy indeksów:

$$\begin{aligned} b, c &\in \{1, \dots, m\}, \\ i &\in \{1, \dots, PathsCount\}. \\ a &\in \{1, \dots, |V(\text{dag}(A))|\}, \\ p &\in \{1, \dots, m+1\}, \\ q &\in \{1, \dots, |A|+1\}, \end{aligned}$$

Zmienna	W etapach	Opis
$cover_{c,i}$	IV	Ścieżka o ID i należy do $covers(c)$.
$prodRF_{p,q}$	Lokalna	Stan (p, q) jest osiągalny w iloczynie automatów.
$subLangF_{b,a}$	Lokalna	Dla identyfikatorów a i b , odpowiadającym komponentom A_a i B_b : $\mathcal{L}(\mathcal{B} \mid B_b) \subseteq \mathcal{L}(\mathcal{A} \mid A_a)$.

Tabela 3.3: Zmienne

Reguły

Zgodnie z algorytmem każdy zbiór $covers$ musi być niepusty. Należy zauważyć (ze względu na poprzedni etap), że komponent istnieje, jeżeli jest niepusty.

$$\bigwedge_{c=1}^m \left(compIsNotEmpty_c \Rightarrow \bigvee_{p=1}^{PathsCount} cover_{c,p} \right) \quad (3a)$$

Przypisywanie ścieżek do zbiorów $covers$.

$$\bigwedge_{c=1}^m \bigwedge_{p=1}^{PathsCount} \left(cover_{c,p} \Rightarrow subLangF_{c, LastNode(p)} \right) \quad (3b)$$

Wybór stanów startowych dla sprawdzania zawierania języków podautomatów.

$$\bigwedge_{c=1}^m \bigwedge_{d=1}^{|V(\text{dag}(A))|} \left(subLangF_{c,d} \Rightarrow \bigvee_{p=1}^m \bigvee_{q \in SccNodes(d)} \left(belongs_{c,p} \wedge \right. \right. \quad (3c)$$

$$\left. \left. \wedge prodRF_{p,q} \right) \right)$$

Przejścia w iloczynie.

$$\begin{aligned} \bigwedge_{p=1}^m \bigwedge_{q=1}^{|A|} \bigwedge_{a=1}^{|\Sigma|} \bigwedge_{k=1}^m & \left(prodRF_{p,q} \wedge t_{p,a,k} \right. \\ & \Rightarrow (sameComponent_{p,k} \Rightarrow prodRF_{k, NextState(q,a)}) \\ & \left. \wedge (\neg sameComponent_{p,k} \Rightarrow prodRF_{sink_B, NextState(q,a)}) \right) \end{aligned} \quad (3d)$$

Stany, które nie mogą być osiągalne (kontrprzykłady zawierania języków):

$$\bigwedge_{q=1}^m \neg \text{prodRF}_{q, \text{sink}_A} \quad (3e)$$

Rozmiar

Podstawowe zmienne są asymptotycznie zdominowane przez zmienne *cover*, których liczba wynosi $O(m|A|^l)$. Reguła (3c) wymaga zastosowania transformacji i wprowadza dodatkowo $O(m^2|A|)$ zmiennych pomocniczych, co nie wpływa na asymptotykę całkowitej liczby zmiennych. Łączna liczba zmiennych wynosi zatem $O(m|A|^l)$. Liczba wystąpień literalów jest asymptotycznie zdominowana przez reguły (3a), (3b) i (3d), i wynosi $O(m|A|^l + m^2|A||\Sigma|)$.

3.1.6. Etap IV: Warunek $\mathcal{B} \subseteq_{\text{ED}}^\infty \mathcal{A}$

Sprawdzanie warunku akceptacji $\mathcal{B} \subseteq_{\text{ED}}^\infty \mathcal{A}$.

Zmienne

Zakresy indeksów:

$$a, b \in \{1, \dots, m\},$$

$$p \in \{1, \dots, \text{PathsCount}\},$$

$$i \in \{1, \dots, \max(1, \lceil \log(\text{PathsCount}) \rceil)\}.$$

Zmienna	W etapach	Opis
$\text{continuationId}_{a,b,p}[i]$	Lokalna	Dla każdej krawędzi (a, b) w DAG-u B , dla każdej ścieżki p w $\text{covers}(a)$, ID zgadywanej ścieżki.

Tabela 3.4: Zmienne

Reguły

Ograniczenie domeny.

$$\bigwedge_{a=1}^m \bigwedge_{b=1}^m \bigwedge_{p=1}^{\text{PathsCount}} (1 \leq \text{continuationId}_{a,b,p} \leq \text{PathsCount}) \quad (4a)$$

Warunek akceptacji.

$$\bigwedge_{a=1}^m \bigwedge_{b=1}^m \bigwedge_{p=1}^{PathsCount} \bigwedge_{d=1}^{PathsCount} (dagE_{a,b} \wedge cover_{a,p} \wedge continuationId_{a,b,p} = d \implies cover_{b,d} \wedge IsPrefix(p, d)) \quad (4b)$$

Rozmiar

Podstawowe zmienne są asymptotycznie zdominowane przez *continuationId*, liczba ich wynosi $O(m^2|A|^l \log |A|)$. Reguła (4a) wymaga zastosowania transformacji i wprowadza dodatkowo $O(m^2|A|^l \log |A|)$ zmiennych pomocniczych, co nie zmienia asymptotyki całkowitej liczby zmiennych. Łączna liczba zmiennych wynosi zatem $O(m^2|A|^l \log |A|)$. Liczba wystąpień literalów jest asymptotycznie zdominowana przez regułę (4b) i wynosi $O(m^2|A|^{2l} \log |A|)$.

Dyskusja

Wprowadzenie zmiennej *continuationId* jest zabiegiem celowym. Bez jej wykorzystania reguła (4b) przyjmuje następującą postać:

$$\bigwedge_{a=1}^m \bigwedge_{b=1}^m \bigwedge_{p=1}^{PathsCount} \left(dagE_{a,b} \wedge cover_{a,p} \implies \bigvee_{d=1}^{PathsCount} (cover_{b,d} \wedge IsPrefix(p, d)) \right)$$

Takie sformułowanie może wydawać się korzystniejsze – nie wymaga wprowadzenia dodatkowych zmiennych. Jednak bezpośrednie przekształcenie powyższej reguły do postaci CNF prowadzi do wykładniczego wzrostu liczby wystąpień literalów. Aby uniknąć tego problemu, konieczne byłoby zastosowanie transformacji Tseytina do podformuły

$$(cover_{b,d} \wedge IsPrefix(p, d))$$

Wymagałoby to wprowadzenia zmiennej pomocniczej parametryzowanej przez b , d oraz p , co wiązałoby się z dodaniem $O(m|A|^{2l})$ zmiennych.

Wprowadzenie zmiennej *continuationId* pozwala uniknąć tego kosztu w procesie konwersji do CNF. Dzięki niej alternatywa występująca wewnątrz formuły może zostać zastąpiona przez koniunkcję umieszczoną na zewnątrz. W rezultacie otrzymujemy formułę, którą można w sposób bezpośredni przetłumaczyć do postaci CNF, ponieważ występująca w niej nierówność ma już postać zgodną z tym formatem. Koszt takiego rozwiązania to $O(m^2|A|^l \log |A|)$ zmiennych.

3.1.7. Etap V: Domknięcie $\text{dag}(B)$

Budowanie domknięcia przechodniego i zwrotnego $\text{dag}(B)$.

Zmienne

Zakresy indeksów: $p, q, s \in \{1, \dots, m\}$

Zmienna	W etapach	Opis
$\text{starDagR}_{p,q,s}$	Lokalna	Dla każdej pary stanów (p, q) w DAG-u, zmienna określająca osiągalność z p do q w co najwyżej s krokach.
$\text{closureE}_{p,q}$	VI	Dla każdej pary stanów (p, q) , zmienna określająca istnienie krawędzi w domknięciu.

Tabela 3.5: Zmienne

Reguły

Jeżeli stan jest osiągalny sam z siebie, to jakiś stan należy do tego komponentu.

$$\bigwedge_{c=1}^m \left(\text{starDagR}_{c,c,1} \implies \bigvee_{p=1}^m \text{belongs}_{c,p} \right) \quad (5a)$$

Jeżeli jakiś stan należy do komponentu, to komponent jest osiągalny sam z siebie.

$$\bigwedge_{c=1}^m \bigwedge_{q=1}^m (\text{belongs}_{c,q} \implies \text{starDagR}_{c,c,1}) \quad (5b)$$

W pierwszym kroku nie da się osiągnąć innych wierzchołków.

$$\bigwedge_{\substack{1 \leq p, q \leq m \\ p \neq q}} \neg \text{starDagR}_{p,q,1} \quad (5c)$$

Rekurencyjne wyznaczanie osiągalności.

$$\bigwedge_{i=1}^m \bigwedge_{j=1}^m \bigwedge_{k=1}^{m-1} \left(\text{starDagR}_{i,j,k+1} \iff \text{starDagR}_{i,j,k} \vee \bigvee_{l=1}^m (\text{dagE}_{i,l} \wedge \text{starDagR}_{l,j,k}) \right) \quad (5d)$$

Przypisanie wyniku osiągalności do domknięcia przechodniego.

$$\bigwedge_{p=1}^m \bigwedge_{q=1}^m (\text{closure}E_{p,q} \iff \text{starDag}R_{p,q,m}) \quad (5e)$$

Rozmiar

Podstawowe zmienne są asymptotycznie zdominowane przez zmienne $\text{starDag}R$, których liczba wynosi $O(m^3)$. Reguła (5d) wymaga zastosowania transformacji i wprowadza dodatkowo $O(m^4)$ zmiennych pomocniczych. Łączna liczba zmiennych wynosi zatem $O(m^4)$. Liczba wystąpień literalów jest asymptotycznie zdominowana przez regułę (5d) i wynosi $O(m^4)$.

3.1.8. Etap VI: Warunek $\mathcal{A} \subseteq_{\text{ED}}^{\infty} \mathcal{B}$

Dla każdej ścieżki z $\text{dag}(A)$ wyszukiwana jest pokrywająca ścieżka z $\text{dag}^*(B)$. Wprowadzona zostaje binarna zmienna parametryzowana przez ID ścieżki i jej długość. Wartość przyjęta przez zmienną reprezentuje wybrany wierzchołek w $\text{dag}^*(B)$. Sprawdzanie zawierania ograniczonych automatów przeprowadzane jest analogicznie do etapu IV.

Zmienne

Zakresy indeksów:

$$\begin{aligned} i &\in \{1, \dots, \text{PathsCount}\}, \\ j &\in \{1, \dots, \text{PathLen}(i)\}, \text{ dla danego } i, \\ k &\in \{1, \dots, \max(1, \lceil \log m \rceil)\}, \\ p &\in \{1, \dots, m+1\}, \\ q &\in \{1, \dots, |A|+1\}, \\ a &\in \{1, \dots, |V(\text{dag}(A))|\}, \\ b &\in \{1, \dots, m\}, \end{aligned}$$

Zmienna	W etapach	Opis
$coveringPath_{i,j}[k]$	Lokalna	Dla każdej ścieżki i w $\text{dag}(A)$, reprezentacja zgadywanej ścieżki pokrywającej w $\text{dag}^*(B)$.
$prodRS_{p,q}$	Lokalna	Dla każdej pary stanów (p, q) w produkcie automatów, zmienna określająca, czy stan (p, q) jest osiągalny.
$subLangS_{a,b}$	Lokalna	Dla identyfikatorów a i b , odpowiadającym komponentom A_a i B_b : $\mathcal{L}(\mathcal{A} \mid A_a) \subseteq \mathcal{L}(\mathcal{B} \mid B_b)$.

Tabela 3.6: Zmienne

Reguły

Ograniczenie domeny.

$$\bigwedge_{p=1}^{PathsCount} \bigwedge_{i=1}^{PathLen(p)} (1 \leq coveringPath_{p,i} \leq m) \quad (6a)$$

Sprawdzanie istnienia wybranej ścieżki.

$$\bigwedge_{p=1}^{PathsCount} \bigwedge_{i=1}^{PathLen(p)-1} \bigwedge_{a=1}^m \bigwedge_{b=1}^m (coveringPath_{p,i} = a \wedge coveringPath_{p,i+1} = b \implies closureE_{a,b}) \quad (6b)$$

Warunek pokrycia: znajdowanie ścieżki pokrywającej.

$$\bigwedge_{p=1}^{PathsCount} \bigwedge_{i=1}^{PathLen(p)} \bigwedge_{c=1}^m \left(coveringPath_{p,i} = c \implies subLangS_{PathNode(p,i),c} \right) \quad (6c)$$

Wybór stanów startowych do sprawdzania zawierania języków podautomatów.

$$\bigwedge_{c=1}^m \bigwedge_{d=1}^{|V(\text{dag}(A))|} \left(subLangS_{d,c} \implies \bigvee_{p=1}^m \bigvee_{q \in \text{ScNodes}(d)} (belongs_{c,p} \wedge prodRS_{p,q}) \right) \quad (6d)$$

Przejścia.

$$\bigwedge_{p=1}^m \bigwedge_{q=1}^{|A|} \bigwedge_{a=1}^{|\Sigma|} \bigwedge_{k=1}^m \left(prodRS_{p,q} \wedge t_{p,a,k} \implies \right. \\ \left. (sameComponent_{p,k} \implies prodRS_{k,NextState(q,a)}) \right. \\ \left. \wedge (\neg sameComponent_{p,k} \implies prodRS_{sink_B,NextState(q,a)}) \right) \quad (6e)$$

Stany zakazane.

$$\bigwedge_{q=1}^{|A|} \neg \text{prodRS}_{\text{sink}_B, q} \quad (6f)$$

Rozmiar

Podstawowe zmienne są asymptotycznie zdominowane przez *coveringPath*. Należy przy tym zauważyć, że $\text{PathLen}(i) \leq l$. Liczba tych zmiennych wynosi zatem $O(|A|^l \log m)$. Reguły (6a) i (6d) wymagają zastosowania transformacji, przy czym reguła (6a) wprowadza dodatkowo $O(|A|^l \log m)$ zmiennych pomocniczych. Nie zmienia to asymptotyki całkowitej liczby zmiennych, która wynosi $O(|A|^l \log m)$. Liczba wystąpień literalów jest asymptotycznie zdominowana przez reguły (6b) i (6e) i wynosi $O(|A|^l m^2 \log m + m^2 |A| |\Sigma|)$.

3.1.9. Analiza asymptotyczna

Etap	Liczba zmiennych	Liczba wystąpień literalów
I	$O(m^2 \Sigma + m^4)$	$O(m^3 \Sigma + m^4)$
II	$O(m^4)$	$O(m^4)$
III	$O(m A ^l)$	$O(m A ^l + m^2 A \Sigma)$
IV	$O(m^2 A ^l \log A)$	$O(m^2 A ^{2l} \log A)$
V	$O(m^4)$	$O(m^4)$
VI	$O(A ^l \log m)$	$O(A ^l m^2 \log m + m^2 A \Sigma)$
Razem	$O(m^2 A ^l \log A + m^2 \Sigma)$	$O(m^2 A ^{2l} \log A + m^2 A \Sigma)$

Tabela 3.7: Podsumowanie liczby zmiennych oraz wystąpień literalów w poszczególnych etapach

Przyjmując zwyczajowo alfabet za stałą, otrzymujemy $O(m^2 |A|^l \log |A|)$ zmiennych i $O(m^2 |A|^{2l} \log |A|)$ literalów.

Przyjmując alfabet jako zmienną, można osiągnąć znaczną poprawę rozmiaru kodowania, stosując zmienne binarne. Koszt ponoszą literalny. Warto zauważyć, że sprawdzanie $\mathcal{A} \subseteq_{\text{ED}}^{\infty} \mathcal{B}$ jest znacznie mniej wymagające niż $\mathcal{B} \subseteq_{\text{ED}}^{\infty} \mathcal{A}$ przy użyciu opisanego algorytmu.

Dyskusja

Algorytm sprawdzający, czy $\mathcal{B} \subseteq_{\text{ED}}^{\infty} \mathcal{A}$, określa warunek akceptacji w następujący sposób: „for all nodes C, C' such that C' is a successor of C in $\text{dag}(B)$, each $\pi \in \text{covers}(C)$ has a continuation in $\text{covers}(C')$ ” [1]. Występują w nim trzy kwantyfikatory uniwersalne. Liczba par (C, C') wynosi $O(m^2)$, natomiast dla każdego zbioru $\text{covers}(C)$ rozpatrywanych jest $O(|A|^l)$ ścieżek. Dla każdej z nich należy sprawdzić istnienie kontynuacji w zbiorze $\text{covers}(C')$, zawierającym również $O(|A|^l)$ ścieżek. Kontynuację można wyszukiwać liniowo lub logarytmicznie, ale w drugim przypadku konieczne jest dodatkowe zastosowanie multipleksera w celu zweryfikowania, czy rozpatrywana ścieżka należy do zbioru. Z powyższych przesłanek wyłania się hipoteza, że w bezpośredniej implementacji tego warunku liczba wymaganych wystąpień literalów wynosi $\Omega(m^2|A|^{2l})$.

3.1.10. Certyfikat

Certyfikat umożliwiający wielomianową weryfikację rozwiązania składa się ze zmiennych **t**, **final**, **belongs**, **cover** oraz **coveringPath**.

Zmienne **t** oraz **final** opisują strukturę automatu wynikowego. Zmienna **origin** pełni natomiast jedynie funkcję pomocniczą i w związku z tym nie musi być przechowywana w certyfikacie. Wszystkie stany, z których możliwe jest osiągnięcie każdej silnie spójnej składowej, mogą stanowić poprawne punkty początkowe. Ich wyznaczenie jest możliwe w czasie liniowym względem rozmiaru automatu.

Zmienna **belongs** określa numerację silnie spójnych składowych oraz przynależność poszczególnych stanów do nich.

Zmienna **cover** określa podzbiór ścieżek przypisanych do danego wierzchołka DAG-u. Z kolei **coveringPath** opisuje ścieżki będące poszczególnymi pokryciami.

Do przeprowadzenia wielomianowej weryfikacji rozwiązania **continuationId** nie jest konieczne. Wynika to z faktu, że każde pokrycie zawiera wielomianową liczbę ścieżek, dzięki czemu identyfikacja kontynuacji może zostać wykonana w czasie wielomianowym również bez dodatkowej informacji zapisanej w certyfikacie. Uwzględnienie zmiennej **continuationId** mogłoby wprowadzić przyspieszyć praktyczny czas weryfikacji poprzez bezpośrednie wskazanie odpowiednich ścieżek, jednak operacja ta nie jest głównym elementem ograniczającym wydajność procesu.

3.1.11. Dyskusja o poprawności i zupełności

Przedstawione kodowanie pozwala na sformułowanie problemu w postaci instancji SAT. Ze względu na ograniczony zakres pracy nie przeprowadzono formalnego dowodu jego poprawności i zupełności.

Poprawność zaproponowanego podejścia została zamiast tego zweryfikowana eksperymentalnie. Dla przygotowanych instancji uzyskane rozwiązania były następnie przekształcane do postaci certyfikatów i poddawane weryfikacji przez program VerifierApp. Wyniki wskazują, że zaproponowane kodowanie poprawnie odwzorowuje rozpatrywany problem dla pozytywnych przypadków.

Formalny dowód poprawności kodowania stanowi naturalny kierunek dalszych prac. Wykazać należy, że każdy model utworzonej formuły SAT odpowiada automatu spełniającemu wymagania problemu, a po drugie, że dla każdego automatu spełniającego te wymagania istnieje odpowiadający mu model formuły. Spełnienie obu tych warunków pozwoliłoby formalnie uzasadnić poprawność i zupełność zaproponowanego rozwiązania.

3.2. Heurystyka generowania automatów o zadanych parametrach

Losowe generowanie DAG-ów stanowi rozbudowany problem [3]. W szczególności trudnym zagadnieniem jest generowanie automatów na ich podstawie, dla którego nie zidentyfikowano odpowiednich opracowań w ramach przeprowadzonego przeglądu literatury. Z tego względu, na potrzeby generowania instancji testowych opracowano następującą heurystykę.

3.2.1. Algorytm

Algorytm 1 Generowanie spójnego DAG-u z jednym korzeniem

Wejście: liczba wierzchołków DAG-u V , liczba stanów automatu N , głębokość DAG-u D .

Wyjście: Wygenerowany DAG.

Ograniczenia: $V \leq N, V \geq D, D = 1 \Rightarrow V = 1$.

Procedura:

- 1: Utwórz D warstw. W pierwszej warstwie umieść wyłącznie korzeń.
- 2: Pozostałe $V - 1$ wierzchołków rozdziel losowo pomiędzy pozostałe $D - 1$ warstw tak, aby każda warstwa zawierała co najmniej jeden wierzchołek.
- 3: Utwórz losowo ścieżkę

$$v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_{D-1}$$

przechodzącą przez wszystkie warstwy.

- 4: Każdemu wierzchołkowi który nie posiada krawędzi wchodzącej, z wyjątkiem korzenia, dodaj losowo krawędzie do wierzchołków znajdujących się w dowolnej wyższej warstwie.
 - 5: Sprawdź, czy pomiędzy V wierzchołków można rozdzielić $N - 1$ stanów w taki sposób, aby każdy wierzchołek v otrzymał co najmniej $\text{outDegree}(v) + 1$ stanów.
 - 6: Jeżeli warunek nie jest spełniony, odrzuć wygenerowany DAG i rozpocznij generowanie od początku. W przeciwnym przypadku zwróć wynik.
-

Algorytm 2 Generowanie automatu na podstawie DAG-u

Wejście: DAG wygenerowany w poprzednim etapie, liczba stanów N , rozmiar alfabetu $|\Sigma|$.

Wyjście: wygenerowany DFA.

Ograniczenia: $|\Sigma| \geq 2$

Procedura:

- 1: Rozdziel $N - 1$ stanów pomiędzy wierzchołki DAG-u. Każdemu wierzchołkowi v przydziel co najmniej $\text{outDegree}(v) + 1$ stanów. Jeden dodatkowy stan jest przeznaczony na globalny stan odrzucający (ang. sink).
 - 2: Dla każdego wierzchołka wygeneruj losowy automat o przydzielonej liczbie stanów, nad alfabetem złożonym z $|\Sigma| - 1$ liter. Losuj automaty aż do uzyskania automatu spełniającego warunek wzajemnej osiągalności słowem, tj. takiego, w którym dla każdej pary stanów istnieje słowo prowadzące z jednego stanu do drugiego.
 - 3: Dla każdego wierzchołka wybierz różne stany odpowiadające jego krawędziom wychodzącym. Każdy wybrany stan przypisz odpowiedniemu dziecku jako punkt połączenia. Połączenie z każdym dzieckiem zrealizuj za pomocą wolnej litery alfabetu.
 - 4: Dla każdego wierzchołka wybierz dodatkowo jeden stan, różny od stanów wybranych w poprzednim kroku, i połącz go ze stanem odrzucającym.
 - 5: Dla pozostałych stanów wylosuj przejście dla wolnej litery. Przejście może prowadzić do:
 - stanu wewnątrz tego samego komponentu,
 - jednego z automatów odpowiadających dzieciom,
 - stanu odrzucającego.
 - 6: Wykonaj losową permutację etykiet wyjściowych dla każdego stanu niezależnie.
 - 7: W każdym liściu DAG-u wylosuj niepusty zbiór stanów akceptujących.
 - 8: W pozostałych automatach wylosuj stany akceptujące.
 - 9: W automacie odpowiadającym korzeniowi wylosuj stan początkowy.
 - 10: Zwróć wygenerowany automat.
-

3.2.2. Własności wygenerowanego automatu

Wygenerowany w ten sposób automat spełnia wymagania dotyczące liczby stanów, liczby wierzchołków DAG-u oraz głębokości SCC.

Ponadto:

- Z każdego stanu da się osiągnąć stan odrzucający, co eliminuje przypadki trywialne (patrz: [1, Proposition 9]).
- Każdy stan jest osiągalny ze stanu początkowego.
- Istnieje tylko jeden stan odrzucający.
- Z każdej silnie spójnej składowej można przejść bezpośrednio do stanu odrzucającego.
- DAG posiada dokładnie jeden korzeń i jest spójny.
- Nie istnieją stany przejściowe.

3.2.3. Ograniczenia metody

Ponieważ każdy wierzchołek wymaga co najmniej $\text{outDegree}(v) + 1$ stanów, metoda skutecznie generuje instancje przede wszystkim wtedy, gdy oczekiwana liczba stanów jest około dwukrotnie większa od oczekiwanej liczby wierzchołków DAG-u. Co więcej, brak stanów przejściowych może znacząco wpływać na wyniki badań dotyczących efektywności minimalizacji, która w warunkach naturalnych, gdzie takie stany występują, mogłaby osiągać znacznie lepsze rezultaty.

Rozdział 4.

Implementacja

W ramach projektu zaimplementowano sześć aplikacji, które realizują poszczególne etapy procesu rozwiązywania problemu lub pełnią funkcję narzędzi pomocniczych.

Do aplikacji głównych należą GenCodingApp, która na podstawie automatu wejściowego oraz docelowej liczby jego stanów generuje plik problemu w postaci kodowania SAT, CertMakerApp, która na podstawie wyniku rozwiązania kodowania SAT generuje certyfikat, VerifierApp, która przyjmuje wygenerowany certyfikat i przeprowadza jego weryfikację pod kątem poprawności. Szczegółowy opis architektury aplikacji przedstawiono w załączniku A.1..

Informacje dotyczące sposobu korzystania z zaimplementowanych aplikacji przedstawiono w załączniku A.2..

4.1. Zastosowane technologie

W trakcie realizacji projektu autor nie znalazł aktywnie utrzymywanych bibliotek charakteryzujących się niewielką liczbą zależności, które mogłyby usprawnić proces implementacji poprzez udostępnienie abstrakcyjnego drzewa składni logicznej, transformacji Tseytina, kodowania formuł do formatu DIMACS [11] oraz ustandaryzowanego mechanizmu mapowania zmiennych na wartości uzyskane z rozwiązania solvera.

Rozwiązania oferujące tego rodzaju funkcjonalności są powszechnie stosowane w silnikach SMT (ang. Satisfiability Modulo Theories) i SAT, jednak ich wykorzystanie wiąże się zazwyczaj z wprowadzeniem dodatkowych, często rozbudowanych zależności. Przykładami takich rozwiązań są solvery Z3 [16] i CaDiCaL [17]. Integracja tego typu narzędzi z projektem byłaby jednak sprzeczna z jednym z jego podstawowych założeń, którym jest uniezależnienie wydajności procesu od wydajności konkretnego solvera. Autor uznał również za nieuzasadnione wprowadzanie do projektu rozbud-

dowanego narzędzia jedynie w celu wykorzystania ograniczonego zakresu oferowanej przez nie funkcjonalności.

Do praktycznej implementacji rozwiązania wykorzystano język C++ w standardzie C++20. Wybór tej technologii został podyktowany przede wszystkim przewidywanymi wymaganiami wydajnościowymi, systemem silnego typowania, który ułatwia rozwój i utrzymanie większych projektów, a także doświadczeniem autora w pracy z tą technologią.

Do automatyzacji procesu budowania aplikacji wykorzystano narzędzie CMake [18], będące powszechnie stosowanym rozwiązaniem w projektach wykorzystujących język C++.

Do przeprowadzenia testów jednostkowych wykorzystano bibliotekę Catch2 [13], wybraną ze względu na prostotę implementacji testów oraz niewielki narzut związany z konfiguracją procesu budowania i zarządzaniem zależnościami.

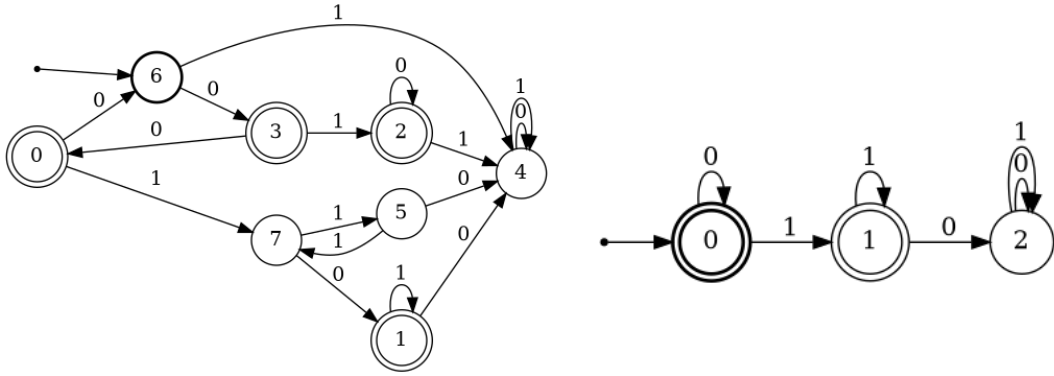
W celu przeprowadzenia wyczerpujących testów (ang. exhaustive testing) wykorzystano solver Togsat [15]. Wybór tego rozwiązania wynikał przede wszystkim z łatwości jego integracji z projektem, braku dodatkowych zależności oraz niewielkiego narzutu związanego z procesem budowania.

Do serializacji i deserializacji danych wykorzystano bibliotekę nlohmann/json [14], która umożliwia szybką i intuicyjną implementację funkcjonalności związanych z formatem.

Zastosowanie powyższych narzędzi pozwoliło ograniczyć liczbę zewnętrznych zależności projektu. W rezultacie projekt charakteryzuje się prostszą konfiguracją, krótszym procesem budowania oraz większą łatwością utrzymania.

Rozdział 5.

Wyniki



Rysunek 5.1: Porównanie struktury automatu przed i po BED-minimalizacji. Automat po lewej ($D = 3$, $V = 4$) jest minimalny w klasycznym sensie.

Eksperymenty przeprowadzano na minimalnych automatach z $|\Sigma| = 2$ spełniających zadane własności. Automaty generowano za pomocą heurystyki, opisanej we wcześniejszej części pracy, do momentu uzyskania instancji, której klasycznie minimalna wersja również spełniała wymagane kryteria.

Na podstawie wstępnych obserwacji taka strategia okazuje się znacznie skuteczniejsza niż jednostajne generowanie automatów aż do uzyskania odpowiedniego obiektu. Ocena jej efektywności stanowi potencjalny kierunek dalszych badań.

Eksperymenty przeprowadzono z wykorzystaniem solvera `kissat 4.0.4` [12] i linuksowego programu `time`. Zmierzono czas wykonania samego programu bez uwzględnienia czasu systemowego. Wyniki czasowe zapisane są w sekundach. Przez współczynnik zmiany rozumie się stosunek wyników, w zależności od przypadku: pomiędzy kolumnami lub wierszami.

Obliczenia wykonano na maszynie o następującej konfiguracji:

System operacyjny: Linux Mint 22.1 x86_64
CPU: AMD Ryzen 7 5700U with Radeon Graphics
GPU: AMD ATI 03:00.0 Lucienne
Pamięć RAM: 15306 MiB

Oznaczenia:

D — głębokość SCC,
 A — liczba stanów automatu wejściowego,
 M — docelowy rozmiar automatu,
 V — liczba wierzchołków DAG-u,
 LK — liczba klauzul,
 LZ — liczba zmiennych.

5.1. Rozmiar formuły SAT

Zmierzono wpływ parametrów na liczbę zmiennych i klauzul w instancji SAT.

Wewnętrzna struktura automatu nie wpływa na wspomniane liczby, tym samym kolejne losowanie dla tych samych parametrów nie zmieniają rezultatu. Dokonywano zatem pojedynczych pomiarów. Łącznie wykonano 15 pomiarów liczby zmiennych i 15 pomiarów liczby klauzul. Wartości przyjęto na podstawie wygenerowanego pliku w formacie DIMACS.

Tabela 5.1: Wpływ liczby wierzchołków DAG-u na rozmiar formuły SAT ($A = 16$, $M = 10$, $D = 3$)

V	LZ	LK	Współczynnik zmiany LZ	Współczynnik zmiany LK
3	89137	282817	–	–
4	102773	317693	1.153	1.123
5	107479	331989	1.046	1.045
6	127115	383665	1.183	1.156
Średnia:			1.127	1.108

Tabela 5.2: Wpływ głębokości SCC na rozmiar formuły SAT ($A = 16$, $M = 10$, $V = 6$)

D	LZ	LK	Współczynnik zmiany LZ	Współczynnik zmiany LK
3	127115	383665	–	–
4	156853	489173	1.234	1.275
5	244749	818109	1.560	1.672
6	302317	1114397	1.235	1.362
Średnia:			1.343	1.437

Tabela 5.3: Wpływ liczby stanów automatu wejściowego na rozmiar formuły SAT ($M = 5$, $D = 3$, $V = 3$)

A	LZ	LK	Współczynnik zmiany LZ	Współczynnik zmiany LK
11	9022	27531	–	–
16	9582	30036	1.062	1.091
21	10142	32541	1.058	1.083
26	10702	35046	1.055	1.077
Średnia:			1.059	1.084

Tabela 5.4: Wpływ rozmiaru docelowego automatu na rozmiar formuły SAT ($A = 21$, $D = 3$, $V = 3$)

M	LZ	LK	Współczynnik zmiany LZ	Współczynnik zmiany LK
5	10142	32541	–	–
10	91247	292822	8.997	8.999
15	400204	1275005	4.386	4.354
20	1198859	3803061	2.996	2.983
Średnia:			5.460	5.445

Obserwacje

Spośród analizowanych parametrów rozmiar docelowy ma zdecydowanie największy wpływ na liczbę zmiennych i klauzul. Istotny wpływ ma również głębokość. Liczba wierzchołków DAG-u wpływa na rozmiar problemu w sposób umiarkowany, natomiast liczba stanów automatu wejściowego okazuje się mieć minimalne znaczenie.

5.2. UNSAT

Zmierzono wpływ parametrów czas potrzebny do zwrócenia wyniku UNSAT. Czas mierzono dla pięciu konfiguracji (A, D, V) : $(20, 3, 5)$, $(30, 3, 5)$, $(30, 3, 10)$, $(30, 4, 5)$ oraz $(30, 5, 5)$. Dla każdej z nich wygenerowano losowo po 4 automaty, a następnie dla każdego z nich zmierzono czas wykonania dla docelowej liczby stanów od 1 do 6. Wyniki uśredniono grupując według (A, D, V, M) . Łącznie przeprowadzono 120 pomiarów.

Tabela 5.5: Wpływ liczby wierzchołków DAG-u na czas uzyskania wyniku UNSAT ($A = 30$, $D = 3$)

M	Śr. pomiar: $V = 5$	Śr. pomiar: $V = 10$	Współczynnik zmiany 5 vs 10
1	0.001	0.001	1.000
2	0.026	0.030	1.154
3	0.047	0.056	1.191
4	0.385	0.416	1.081
5	1.824	2.612	1.432
6	28.438	24.750	0.870
Średnia:			1.121

Tabela 5.6: Wpływ głębokości SCC na czas uzyskania wyniku UNSAT ($A = 30$, $V = 5$)

M	Śr. pomiar: $D = 3$	Śr. pomiar: $D = 4$	Śr. pomiar: $D = 5$	Współczynnik 3 vs 4	Współczynnik 4 vs 5
1	0.001	0.001	0.002	1.000	2.000
2	0.026	0.027	0.029	1.038	1.074
3	0.047	0.049	0.056	1.043	1.143
4	0.385	0.383	0.451	0.995	1.178
5	1.824	2.158	2.312	1.183	1.071
6	28.438	25.993	22.412	0.914	0.862
Średnia:				1.029	1.221
Średnia całkowita:				1.125	

Tabela 5.7: Wpływ rozmiaru docelowego automatu na czas uzyskania wyniku UNSAT ($A = 30$, $D = 3$, $V = 5$)

M	Śr. pomiar	Współczynnik zmiany
1	0.001	–
2	0.026	26.000
3	0.047	1.808
4	0.385	8.191
5	1.824	4.738
6	28.438	15.591
Średnia:		11.266

Tabela 5.8: Wpływ rozmiaru automatu na czas uzyskania wyniku UNSAT ($D = 3$, $V = 5$)

M	Śr. pomiar: $A = 20$	Śr. pomiar: $A = 30$	Współczynnik zmiany 20 vs 30
1	0.003	0.001	0.333
2	0.027	0.026	0.963
3	0.043	0.047	1.093
4	0.288	0.385	1.337
5	1.917	1.824	0.951
6	17.351	28.438	1.639
Średnia:			1.053

Obserwacje

Zwiększenie rozmiaru docelowego ma dominujące znaczenie dla czasu uzyskania wyniku UNSAT. Głębokość ma podobny wpływ co liczba wierzchołków. Liczba stanów ma znikomy wpływ.

5.3. Instancje optymalne i graniczne

Zmierzono wpływ parametrów na czas rozwiązania instancji optymalnych i instancji granicznych.

Przez instancję optymalną rozumie się instancję, dla której dobrana liczba stanów automatu docelowego jest równa liczbie stanów BED-zminimalizowanego automatu.

Przez instancję graniczną rozumie się instancję, dla której dobrana liczba stanów automatu docelowego jest o jeden mniejsza niż liczba stanów BED-zminimalizowanego automatu.

Dla konfiguracji parametrów (A, D, V) : $(6, 3, 3)$, $(7, 3, 3)$, $(8, 3, 3)$, $(8, 3, 4)$, $(8, 4, 4)$ oraz $(9, 3, 3)$ zmierzono liczbę stanów zminimalizowanego automatu, czas rozwiązania instancji optymalnej oraz czas rozwiązania instancji granicznej. Dla konfiguracji $(9, 3, 4)$ oraz $(9, 4, 4)$ zmierzono wyłącznie liczbę stanów zminimalizowanego automatu. W ramach każdej z konfiguracji wygenerowano losowo po 4 automaty, a wyniki uśredniono w grupach (A, D, V) . Łącznie przeprowadzono 24 pomiary czasu dla instancji optymalnych, 24 pomiary dla instancji granicznych oraz 32 pomiary liczby stanów zminimalizowanego automatu.

Oznaczenie SAT odnosi się do czasu potrzebnego na rozwiązanie instancji optymalnej, natomiast UNSAT określa czas rozwiązania instancji granicznej.

Tabela 5.9: Wpływ głębokości SCC na czas minimalizacji ($V = 4$, $A = 8$)

D	Średni pomiar SAT	Średni pomiar UNSAT	Współczynnik zmiany SAT	Współczynnik zmiany UNSAT
3	0.669	1.272	–	–
4	0.590	2.032	0.882	1.597

Tabela 5.10: Wpływ liczby wierzchołków DAG-u na czas minimalizacji ($D = 3$, $A = 8$)

V	Średni pomiar SAT	Średni pomiar UNSAT	Współczynnik zmiany SAT	Współczynnik zmiany UNSAT
3	0.324	1.077	–	–
4	0.669	1.272	2.066	1.181

Tabela 5.11: Wpływ liczby stanów automatu wejściowego na czas minimalizacji ($D = 3$, $V = 3$)

A	Średni pomiar SAT	Średni pomiar UNSAT	Współczynnik zmiany SAT	Współczynnik zmiany UNSAT
6	0.117	0.179	–	–
7	0.195	0.523	1.672	2.922
8	0.324	1.077	1.658	2.059
9	1.028	4.060	3.174	3.770
		Średnia:	2.168	2.917

Tabela 5.12: Stanowa efektywność minimalizacji dla różnych parametrów

D	V	A	Śr. redukcja liczby stanów
3	3	6	20.8%
		7	35.7%
		8	28.1%
		9	33.3%
Średnia całkowita:			29.5%
3	4	8	28.1%
		9	22.2%
Średnia całkowita:			25.2%
4	4	8	31.3%
		9	33.3%
Średnia całkowita:			32.3%

Obserwacje

Liczba stanów i liczba wierzchołków mają dominujące i podobne znaczenie dla czasu rozwiązania optymalnej instancji. Należy pamiętać, że im wyższa liczba stanów, tym większy zakres możliwej wartości stanu docelowego, dlatego nie należy tych wyników interpretować w oderwaniu od obserwacji z innych części. Zwiększanie głębokości wykazało redukcję czasu potrzebnego na rozwiązanie instancji optymalnej.

Liczba stanów ma dominujący wpływ na czas rozwiązania granicznej instancji (uwaga, patrz wyżej). W drugiej kolejności występuje głębokość. Szczególnie wyróżnia się liczba wierzchołków, która ma niski wpływ na czas rozwiązania granicznej instancji, w przeciwieństwie do optymalnej.

Dla badanych konfiguracji, efektywność w ilości minimalizacji stanów kształtuje się na poziomie około 30%, przy czym obserwowane wartości mieszczą się w zakresie od około 22% do 36%.

5.4. Wnioski

Docelowy rozmiar automatu ma dominujący wpływ na trudność problemu. Wskazuje na to zarówno analiza UNSAT, analiza liczby zmiennych i klauzul, jak i pośrednio analiza instancji optymalnych i granicznych. Zwiększanie głębokości wywiera umiarkowany wpływ na analizowane wyniki. Drugim pod względem znaczenia parametrem jest liczba wierzchołków. Liczba stanów automatu wejściowego wykazuje w praktyce znikome znaczenie. Negatywny wynik instancji okazuje się w przypadku tego problemu o wiele trudniejszy do uzyskania niż wynik pozytywny. Efektywność w redukcji stanów kształtuje się na poziomie około 30%. Problem w ogólności okazuje się trudny już dla wartości M w okolicach 10. Trudne instancje występują już dla wartości 8 i 9. Podczas pracy nad analizą wygenerowano instancję $A = 10$, $D = 3$, $V = 3$, $M = 8$, dla której solver zwrócił wynik UNSAT po 35 minutach. W przypadku obranej heurystyki tylko dwa razy w trakcie pomiarów wygenerowano automaty, dla których udało się potwierdzić, że nie da się ich zminimalizować w sensie BED, co wskazuje na potencjalnie wysoką szansę powodzenia minimalizacji. Liczba wierzchołków DAG-u silnie obciąża czas rozwiązania instancji optymalnej, lecz ma niski wpływ na instancję graniczną. Głębokość nie wpłynęła na wydłużenie czasu potrzebnego do rozwiązania instancji optymalnej.

Rozdział 6.

Podsumowanie

W ramach pracy opracowano pierwsze, nietrywialne kodowanie problemu MIN-BED do SAT. Zaproponowano heurystykę generowania automatów o zadanych parametrach, umożliwiającą badanie charakterystyki różnych klas instancji. Opracowano obszerną implementację, obejmującą m.in. generowanie automatów pseudolosowych, kodowanie instancji problemu do formatu SAT, generowanie certyfikatów rozwiązań oraz ich weryfikację. Przeprowadzone eksperymenty dostarczyły pierwszych usystematyzowanych wyników dotyczących problemu, pozwalając na wstępną analizę jego właściwości oraz specyfiki obliczeniowej w poszczególnych przypadkach.

Kwestie otwarte obejmują m.in. dalsze badania eksperymentalne, optymalizację kodowania z uwzględnieniem dotychczasowych wniosków empirycznych, opracowanie metod generowania automatów spełniających określone ograniczenia, a także implementację szczególnych rozwiązań wielomianowych w oparciu o powstałą implementację.

Opracowane rezultaty tworzą tym samym podstawę do dalszych badań nad problemem MIN-BED, zarówno w aspekcie eksperymentalnym, jak i teoretycznym.

Bibliografia

- [1] J. Michaliszyn, J. Otop, *Minimization of Deterministic Finite Automata Modulo the Edit Distance*. W: *50th International Symposium on Mathematical Foundations of Computer Science (MFCS 2025)*. doi:10.4230/LIPIcs.MFCS.2025.77.
- [2] P. Gawrychowski, A. Jeż, *Hyper-minimisation Made Efficient*. W: *MFCS 2009*, s. 356–368, 2009. doi:10.1007/978-3-642-03816-7_31.
- [3] J. Kuipers, G. Moffa, *Uniform random generation of large acyclic digraphs*, *Statistics and Computing*, t. 25, nr 2, s. 227–242, 2015. doi:10.1007/s11222-013-9428-y.
- [4] G. S. Tseytin, *On the complexity of derivation in propositional calculus*. W: Slisenko, A. O. (red.), *Studies in Constructive Mathematics and Mathematical Logic, Part II*, *Seminars in Mathematics*, Steklov Mathematical Institute (1970).
- [5] E. Zhu, S. Huang, S. Chaudhuri, *High-performance row pattern recognition using joins*, *Proc. VLDB Endow.*, t. 16, nr 5, s. 1181–1195, styczeń 2023. doi:10.14778/3579075.3579090.
- [6] A. Bremler-Barr, D. Hay, Y. Koral, *CompactDFA: Scalable pattern matching using longest prefix match solutions*, *IEEE/ACM Transactions on Networking*, t. 22, nr 2, s. 415–428, 2013. doi:10.1109/TNET.2013.2253119.
- [7] M. de Leoni, P. Felli, M. Montali, *Strategy Synthesis for Data-Aware Dynamic Systems with Multiple Actors*, W: *Proceedings of the 17th International Conference on Principles of Knowledge Representation and Reasoning*, s. 315–325, wrzesień 2020. doi:10.24963/kr.2020/32.
- [8] D. Aineto, S. Jimenez, E. Onaindia, *Generalized Temporal Inference via Planning*, W: *Proceedings of the 18th International Conference on Principles of Knowledge Representation and Reasoning*, s. 22–31, listopad 2021. doi:10.24963/kr.2021/3.
- [9] E. Roche, Y. Schabes, *Deterministic part-of-speech tagging with finite state transducers*, *Computational Linguistics*, t. 21, nr 2, s. 227–253, 1995.

- [10] D. Carmel, S. Markovitch, *Opponent modeling in multi-agent systems*, W: International Joint Conference on Artificial Intelligence, s. 40–52, Springer, 1995. doi:10.1007/3-540-60923-7_18.
- [11] Jannis Harder, *Varisat manual - DIMACS CNF* <https://jix.github.io/varisat/manual/0.2.0/formats/dimacs.html> (dostęp: 1 września 2026).
- [12] A. Biere, *Kissat*, repozytorium: <https://github.com/arminbiere/kissat> (dostęp: 1 września 2026).
- [13] Catch Org, *Catch2*, repozytorium: <https://github.com/catchorg/Catch2> (dostęp: 1 września 2026).
- [14] N. Lohmann, *nlohmann/json*, repozytorium: <https://github.com/nlohmann/json> (dostęp: 1 września 2026).
- [15] H. Togasaki, *Togasat*, repozytorium: <https://github.com/togatoga/Togasat> (dostęp: 1 września 2026).
- [16] Microsoft Research, *Z3*, repozytorium: <https://github.com/Z3Prover/z3> (dostęp: 1 września 2026).
- [17] A. Biere, *CaDiCaL*, repozytorium: <https://github.com/arminbiere/cadical> (dostęp: 1 września 2026).
- [18] Kitware Inc, *CMake*, strona narzędzia <https://cmake.org/> (dostęp: 1 września 2026).

Dodatek A

Załącznik - szczegóły implementacji

A.1. Architektura systemu

A.1.1. Struktura projektu

Struktura projektu została podzielona na moduły, z których każdy odpowiada za określony zakres funkcjonalności:

- **AugScc** – przyjmuje automat wejściowy i na jego podstawie tworzy graf silnie spójnych składowych (SCC) spełniający wymagane własności. Moduł zapewnia również dostęp do zasobów umożliwiających sprawne wykorzystanie wyników przeprowadzonych prekalkulacji podczas generowania kodowania.
- **CodeGen** – odpowiada za generowanie kodowania na podstawie dowolnego obiektu spełniającego wymagania dotyczące dostępu do wyników prekalkulacji, w szczególności obiektu z **AugScc**. Moduł uwzględnia również zadaną maksymalną liczbę stanów automatu wynikowego. Wygenerowane kodowanie może zostać zapisane do pliku.
- **Logic** – podmoduł **CodeGen** odpowiedzialny za dostarczanie narzędzi niezbędnych do efektywnego przygotowania kodowania. Udostępnia między innymi implementacje wektorów bitowych, operacji pseudoboolowskich, struktur reprezentujących formuły CNF, mechanizmy konwersji formuł do formatu DIMACS, składnię wyrażeń logicznych oraz implementację transformacji Tseytina.
- **CertReader** – przyjmuje wynik zwrócony przez solver wraz z pierwotnym przypisaniem zmiennych i przetwarza go do uporządkowanej struktury wynikowej.

- **Verifier** – odpowiada za weryfikację poprawności otrzymanego certyfikatu. Stanowi dodatkową warstwę zabezpieczającą przed błędami implementacyjnymi, w szczególności przed wystąpieniem wyników fałszywie pozytywnych. Moduł ułatwia również proces debugowania oraz umożliwia niezależną od procesu znajdowania rozwiązania weryfikację otrzymanego wyniku.
- **Serializer** – odpowiada za serializację i deserializację obiektów do formatu JSON. Moduł umożliwia również zapis automatu deterministycznego (DFA) w formacie `.dot`, pozwalającym na jego graficzną wizualizację za pomocą odpowiedniego oprogramowania, takiego jak Graphviz.
- **DfaGen** – generuje pseudolosowe automaty o zadanej liczbie stanów, rozmiarze alfabetu, liczbie silnie spójnych składowych oraz określonej głębokości SCC. Moduł ten jest wykorzystywany do generowania danych testowych.
- **Utils** – zawiera pomocnicze struktury i funkcje wykorzystywane globalnie przez moduły projektu.
- **Shared** – zawiera definicje współdzielonych struktur i kontraktów definiowanych w plikach nagłówkowych.
- **BedMinimizer** – stanowi warstwę integrującą pozostałe moduły i udostępnia uproszczony interfejs, ukrywając szczegóły implementacji.

A.1.2. Programy użytkowe

W ramach projektu zaimplementowano sześć aplikacji, które realizują poszczególne etapy procesu lub pełnią funkcję narzędzi pomocniczych.

Aplikacje główne:

- **GenCodingApp** – przyjmuje automat w formacie JSON i docelową liczbę stanów automatu, który ma zostać znaleziony. Na podstawie podanych danych generuje plik problemu w postaci kodowania SAT i plik instancji wymagany na dalszym etapie.
- **CertMakerApp** – przyjmuje wynik rozwiązania kodowania SAT i ścieżkę do pliku z informacjami o pierwotnej instancji. Na podstawie tych danych generuje certyfikat w formacie JSON.
- **VerifierApp** – przyjmuje wygenerowany certyfikat i przeprowadza jego weryfikację pod kątem poprawności.

Aplikacje pomocnicze:

- **DfaGenApp** – przyjmuje jako argumenty liczbę stanów docelowego automatu, liczbę silnie spójnych składowych (SCC), ich maksymalną głębokość, rozmiar alfabetu oraz nazwę pliku wyjściowego. Na tej podstawie generuje automat w formacie JSON spełniający zadane wymagania. W przypadku niepowodzenia generowania po wykonaniu określonej liczby prób aplikacja zwraca błąd.
- **ClassicMinimizeApp** – przyjmuje automat w formacie projektu i minimalizuje go za pomocą klasycznej metody.
- **DfaToDotApp** – przyjmuje automat w formacie projektu i konwertuje go do formatu `.dot`.

A.1.3. Format pliku DFA

Automat reprezentowany jest jako obiekt JSON zawierający liczbę stanów, rozmiar alfabetu, stan początkowy, zbiór stanów akceptujących oraz funkcję przejścia.

Przykładowa reprezentacja automatu ma postać:

```
{
  "states_count": 3,
  "alphabet_size": 2,
  "initial_state": 0,
  "accept_states": [2],
  "transitions": [
    [1, 0],
    [2, 1],
    [2, 0]
  ]
}
```

Pole `transitions` zawiera tablicę przejść, w której każdy wiersz odpowiada jednemu stanowi, a każda kolumna odpowiada symbolowi wejściowemu.

Przykładowo pierwszy element tablicy `transitions`, znajdujący się pod indeksem 0, ma postać `[1, 0]`. Oznacza to, że $\delta(0, 0) = 1$ oraz $\delta(0, 1) = 0$. Analogicznie kolejny wiersz `[2, 1]` oznacza, że $\delta(1, 0) = 2$ oraz $\delta(1, 1) = 1$.

A.2. Sposób użytkowania

Proces wykorzystania zaimplementowanego rozwiązania rozpoczyna się od przekazania do programu **GenCodingApp** automatu przeznaczonego do minimalizacji oraz docelowej liczby stanów automatu wynikowego. W rezultacie generowane są dwa pliki: plik zawierający kodowanie problemu w standardowym formacie przeznaczonym dla solverów SAT oraz plik zawierający dodatkowe informacje dotyczące danej instancji.

Plik z informacjami o instancji zawiera dane niezbędne do późniejszego wygenerowania wyniku, których nie można w prosty sposób odtworzyć wyłącznie na podstawie automatu wejściowego. W szczególności obejmuje on automat wejściowy, numerację oraz zawartość silnie spójnych składowych, numerację i opis ścieżek, a także mapowanie zmiennych z formatu DIMACS na odpowiadające im elementy modelu, umożliwiające określenie znaczenia poszczególnych zmiennych.

Po znalezieniu przez solver pozytywnego rozwiązania należy przekazać plik zawierający wynik działania solvera z flagą `-q` oraz odpowiadający mu plik z informacjami o instancji do programu **CertMakerApp**. Na ich podstawie aplikacja generuje certyfikat rozwiązania i automat wynikowy.

Wygenerowany certyfikat może następnie zostać poddany weryfikacji za pomocą programu **VerifierApp**.